AI Agent Harness: оркестрация (кратко для ментора)

Nov 7, 2026 · @Sergey Kashkov

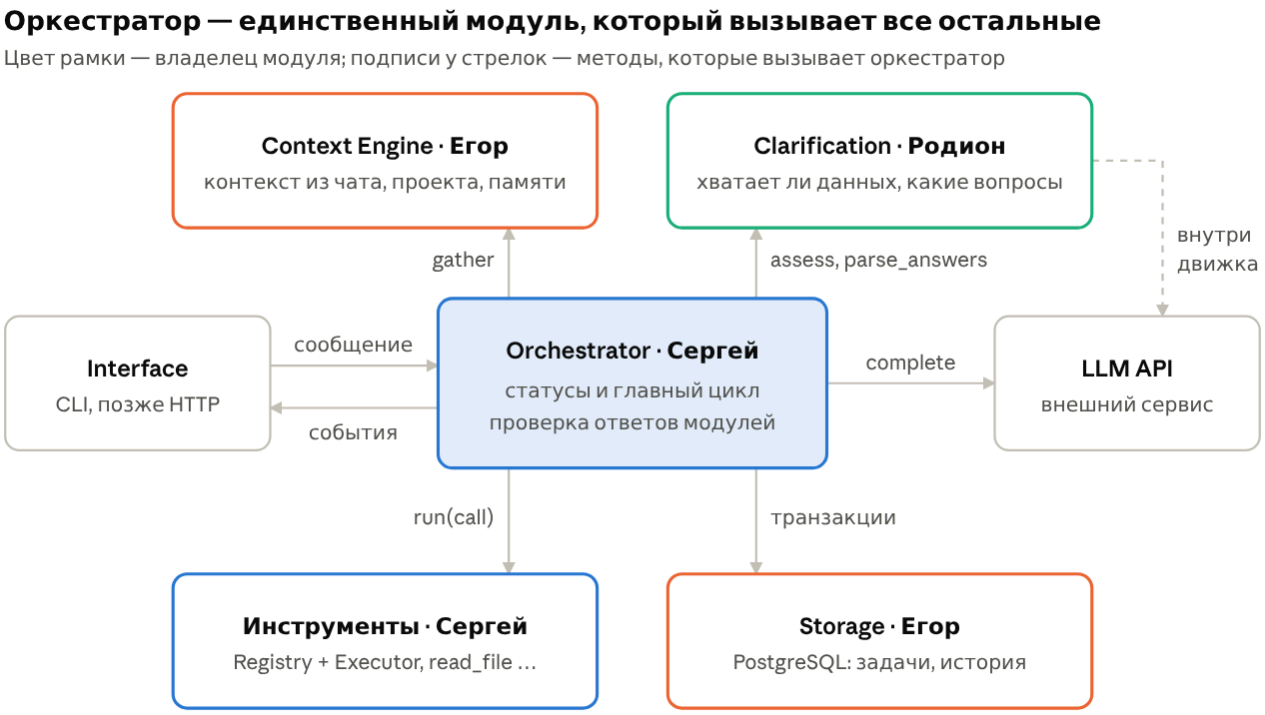
Задача

Мы делаем harness — обвязку вокруг готовой LLM, которая не даёт модели додумывать критичные параметры задачи.

- **Проблема.** На запрос «Проверь тесты проекта» при двух проектах в чате модель сама выбирает проект и может ошибиться.
- **Решение.** Сначала искать недостающее в контексте, если не нашлось — задать 1–3 уточняющих вопроса, и только потом выполнять. Некритичное можно принять по умолчанию, но явно написать об этом в ответе.
- **Рамки MVP.** Модель не обучаем. Единственный специальный модуль — уточняющие вопросы. Стек: Python, Pydantic, PostgreSQL, собственный конечный автомат без LangChain.

Архитектура

Шесть модулей и внешняя LLM. Модули не знают друг о друге: все вызовы идут через оркестратор, а общаются они через заранее согласованные интерфейсы. Это позволяет троим разрабатывать свои части параллельно, с заглушками вместо чужих модулей.



компоненты harness и владельцы

Участник	Модули
Сергей	Orchestrator; инструменты, LLM-адаптер и интерфейс (предложение)
Егор	Storage (PostgreSQL); Context Engine (предложение)
Родион	Clarification Engine — единственный специальный skill

Как обрабатывается запрос

Пример: «Проверь тесты проекта», к чату привязаны `backend` и `frontend`. Задача проходит статусы `new → analyzing → waiting_user → analyzing → executing → completed`.

1. **Анализ.** Оркестратор создаёт задачу, собирает контекст и спрашивает Clarification Engine, хватает ли данных. Ответ: нет, неизвестен проект.
2. **Вопрос.** Пользователь получает «Какой проект проверить? (backend, frontend)». Задача сохранена в базе и ждёт ответа сколько угодно. LLM для выполнения ещё не вызывалась.

3. **Ответ.** Пользователь пишет «backend». Clarification Engine разбирает ответ, и в задачу записывается `project_id = backend` с пометкой «сказал пользователь».
4. **Повторный анализ.** Данных хватает. Одно допущение принято явно: «проверяю все тесты».
5. **Выполнение.** Оркестратор вызывает LLM со списком инструментов. Модель просит `read_file("README.md")`, оркестратор выполняет и возвращает результат. Цикл повторяется, пока модель не ответит текстом.
6. **Ответ пользователю** — результат и блок «Допущения».

Если во время выполнения модель обнаружит новую критичную неопределённость, она вызывает псевдоинструмент `request_clarification`, и задача возвращается к шагу анализа.

Моя часть: оркестратор

Оркестратор ведёт задачу от запроса до ответа и решает, какой модуль вызвать следующим. Ключевые решения:

Решение	Зачем
Конечный автомат из 7 статусов с таблицей разрешённых переходов (`new`, `analyzing`, `waiting_user`, `executing`, `completed`, `failed`, `cancelled`)	по статусу всегда понятно, что делать дальше; недопустимый переход — сразу ошибка
Всё состояние в базе, запись после каждого шага	задача может ждать ответа сколько угодно и продолжается после падения процесса (`resume`)
Долгие вызовы — вне транзакций: прочитать → вызвать LLM / инструмент → записать одной транзакцией	транзакции короткие, промежуточных состояний нет
Версия задачи и оптимистичная блокировка	отмена во время работы не теряется, два процесса не ведут одну задачу
Проверка ответа Clarification Engine по 7 правилам (не больше 3 вопросов, не спрашивать известное, допущение не закрывает критичный пробел …)	движок использует LLM и может ошибаться, а главное правило проекта нарушать нельзя

Решение	Зачем
Допущения добавляет в ответ оркестратор, а не модель	модель может забыть о них написать
Лимиты: 3 раунда уточнения, 12 шагов, 30 вызовов инструментов, 200 000 токенов, запрет третьего одинакового вызова	модель и движок уточнения не могут зациклиться; по исчерпании лимита — честный отказ вместо догадки
Ошибки инструментов возвращаются модели, временные ошибки LLM повторяются с паузой	один неудачный вызов не роняет задачу
Защита от дублей по `client_message_id`	повторная отправка сообщения не создаёт вторую задачу

Что я прошу у коллег

Оркестратор пишется против этих интерфейсов. Подробные запросы с сигнатурами и примерами отправлены Егору и Родиону отдельно.

Кому	Интерфейс	Главные требования
Егор	`Storage.unit_of_work()` + репозитории: задачи, сообщения, вопросы/ответы, вызовы инструментов, снимки, проекты, inbox	атомарные транзакции; `tasks.update` с проверкой версии; одна активная задача на чат; порядок сообщений сохраняется
Егор	`ContextEngine.gather(query) → ContextBundle`	только разрешённые источники; размер не больше бюджета; у каждого блока есть ссылка на источник и SHA-256
Родион	`assess(input) → ASK / PROCEED / ABORT`	до 3 вопросов по приоритету; допущения только для некритичного; не пишет в базу
Родион	`parse_answers(input) → ANSWER / CANCEL / NEW_REQUEST / UNCLEAR`	ответы только на заданные вопросы; распознаёт отмену и новый запрос

Статус и план

Сделано: спроектированы оркестрация и интерфейсы между модулями, коллегам отправлены запросы API. Есть прототип оркестратора на заглушках: сценарий «Проверь тесты проекта» с уточнением проходит целиком, 38 тестов (отмена, лимиты, восстановление после сбоя, ошибки LLM).

Дальше:

1. Согласовать интерфейсы с Егором и Родионом и зафиксировать их.
2. Параллельная разработка: Storage на PostgreSQL, Clarification Engine на LLM, адаптер к реальной модели и консольный интерфейс.
3. Интеграция: заменить заглушки настоящими модулями, все тесты должны проходить.
4. Набор из 15–20 реальных сценариев, подбор лимитов и весов приоритета вопросов, метрики (доля задач с уточнением, число раундов, причины отказов).

Вопросы к ментору

- Оставить собственный конечный автомат или сразу брать LangGraph?
- Какую LLM и какой SDK использовать?
- Как оценивать качество уточняющих вопросов: какие метрики и какой набор сценариев считать достаточным?
- Нормально ли в MVP ограничиться одной активной задачей на чат и отклонять сообщения во время выполнения?
- Добавляем ли статус `cancelled` в официальный MVP-документ?